LeetCode HOT 100

PART 9

Graph

图论

51. 岛屿数量 (No.200) MEDIUM

给你一个由 `'1'` （陆地）和 `'0'` （水）组成的的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

示例 1：

```
输入：grid = [
  ['1','1','1','1','0'],
  ['1','1','0','1','0'],
  ['1','1','0','0','0'],
  ['0','0','0','0','0']
]
输出：1
```

示例 2：

```
输入：grid = [
  ['1','1','0','0','0'],
  ['1','1','0','0','0'],
  ['0','0','1','0','0'],
  ['0','0','0','1','1']
]
输出：3
```

提示：

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` 的值为 `'0'` 或 `'1'`

51. 岛屿数量 (No.200) MEDIUM

思路

遍历grid，遇到'1'就开始深度优先搜索，把'1'周围的'1'全部改为'0'，就像从小岛某一处开始淹没整个岛，之后遍历就不会算重复了。dfs的边界条件是下标越界或遇到海面。

时间复杂度：$\mathcal{O}(mn)$

空间复杂度：$\mathcal{O}(mn)$

```
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):  # 从grid[i][j]处开始淹没小岛
            # 边界条件
            if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] == '0':
                return
            grid[i][j] = '0'
            dfs(i-1, j)
            dfs(i+1, j)
            dfs(i, j-1)
            dfs(i, j+1)

        m, n = len(grid), len(grid[0])
        res = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    res += 1
                    dfs(i, j)
        return res
```

52. 腐烂的橘子 (No.994) MEDIUM

在给定的 `m x n` 网格 `grid` 中，每个单元格可以有以下三个值之一：

- 值 `0` 代表空单元格；
- 值 `1` 代表新鲜橘子；
- 值 `2` 代表腐烂的橘子。

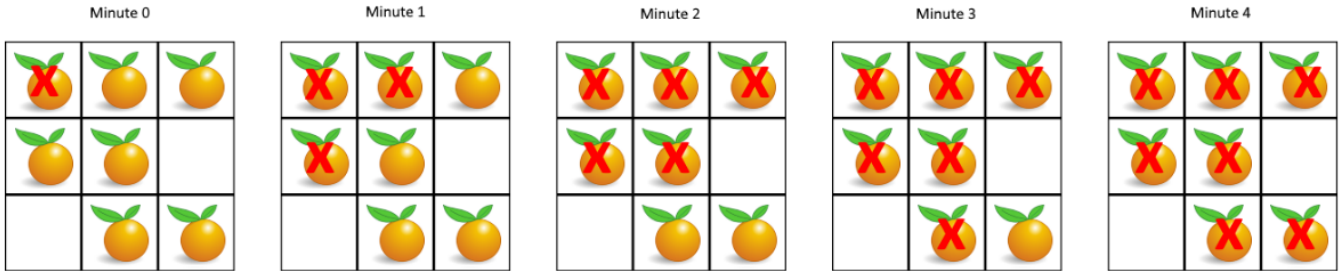
每分钟，腐烂的橘子 **周围 4 个方向上相邻** 的新鲜橘子都会腐烂。

返回 *直到单元格中没有新鲜橘子为止所必须经过的最小分钟数。如果不可能，返回* `-1` 。

提示：

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 10`
- `grid[i][j]` 仅为 `0`、`1` 或 `2`

示例 1：



> **输入：** `grid = [[2,1,1],[1,1,0],[0,1,1]]`
> **输出：** `4`

示例 2：

> **输入：** `grid = [[2,1,1],[0,1,1],[1,0,1]]`
> **输出：** `-1`
> **解释：** 左下角的橘子（第 2 行， 第 0 列）永远不会腐烂，因为腐烂只会发生在 4 个方向上。

示例 3：

> **输入：** `grid = [[0,2]]`
> **输出：** `0`
> **解释：** 因为 0 分钟时已经没有新鲜橘子了，所以答案就是 0 。

52. 腐烂的橘子 (No.994) MEDIUM

思路

多源广度优先搜索，初始queue保存所有腐烂的橘子，fresh记录所有新鲜的橘子。如果queue为空说明没有刚腐烂的橘子，fresh为空说明场上没有新鲜的橘子，这两种情况都应该退出循环。

每轮处理刚腐烂的橘子，这些橘子的邻居如果是新鲜橘子都要腐烂，然后入队并从fresh中删除。最终如果fresh还有新鲜橘子返回-1，否则就返回循环的轮数。

时间复杂度：$\mathcal{O}(mn)$

空间复杂度：$\mathcal{O}(mn)$

```python
class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        def rot(i, j):
            # 如果i, j合法且对应新鲜橘子，将橘子腐烂并入队
            if 0 <= i < m and 0 <= j < n and grid[i][j] == 1:
                grid[i][j] = 2
                queue.append((i, j))
                fresh.remove((i, j))

        m, n = len(grid), len(grid[0])
        queue, fresh = deque(), set()
        # 找到所有腐烂和新鲜的橘子，分别装进queue和fresh
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 2:
                    queue.append((i, j))
                if grid[i][j] == 1:
                    fresh.add((i, j))
        # 多源广度优先搜索
        res = 0
        while queue and fresh:
            q_len = len(queue)
            for _ in range(q_len):
                i, j = queue.popleft()
                # 周边的橘子腐烂并入队
                rot(i-1, j)
                rot(i+1, j)
                rot(i, j-1)
                rot(i, j+1)
            res += 1
        # 检查还有没有新鲜橘子
        if fresh:
            res = -1
        return res
```

53. 课程表 (No.207) MEDIUM

你这个学期必须选修 `numCourses` 门课程，记为 `0` 到 `numCourses - 1` 。

在选修某些课程之前需要一些先修课程。 先修课程按数组 `prerequisites` 给出，其中 `prerequisites[i] = [a_i, b_i]` ，表示如果要学习课程 a_i 则 **必须** 先学习课程 b_i 。

- 例如，先修课程对 `[0, 1]` 表示：想要学习课程 `0` ，你需要先完成课程 `1` 。

请你判断是否可能完成所有课程的学习？如果可以，返回 `true` ；否则，返回 `false` 。

示例 1：

> **输入：** numCourses = 2, prerequisites = [[1,0]]
> **输出：** true
> **解释：** 总共有 2 门课程。学习课程 1 之前，你需要完成课程 0 。这是可能的。

示例 2：

> **输入：** numCourses = 2, prerequisites = [[1,0],[0,1]]
> **输出：** false
> **解释：** 总共有 2 门课程。学习课程 1 之前，你需要先完成课程 0 ；并且学习课程 0 之前，你还应先完成课程 1 。这是不可能的。

提示：

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= a_i, b_i < numCourses`
- `prerequisites[i]` 中的所有课程对 **互不相同**

53. 课程表 (No.207) MEDIUM

Solution 1 – 深度优先

思路

三色标记法，color=0表示该节点尚未访问，color=1表示当前dfs递归中正在访问，color=2表示已经访问过。

用邻接表graph存储图，只需要从每个点出发dfs，判断是否成环，成环则无法修读完所有课程，反之一定能修完。

时间复杂度：$\mathcal{O}(|V|+|E|)$

空间复杂度：$\mathcal{O}(|V|+|E|)$

V是顶点集，E是边集

```python
class Solution(object):
    def canFinish(self, numCourses, prerequisites):
        """
        :type numCourses: int
        :type prerequisites: List[List[int]]
        :rtype: bool
        """
        def dfs(course):
            # 课程编号为course的课程深度优先搜索，成环返回True，否则False
            colors[course] = 1  # 标记当前dfs正在访问
            for post in graph[course]:
                # 当前步成环或post继续递归成环
                if colors[post] == 1 or colors[post] == 0 and dfs(post):
                    return True
            colors[course] = 2  # 访问完毕
            return False

        # graph中key=课程编号,value=key的后修课程列表
        graph = defaultdict(list)
        for pre in prerequisites:
            graph[pre[1]].append(pre[0])
        # color=0:节点尚未访问; =1:当前dfs正在访问; =2: 已访问
        colors = [0] * numCourses
        # 从每个节点开始遍历，只需要找是否成环
        for i in range(numCourses):
            if colors[i] == 0 and dfs(i):
                return False
        return True
```

TIPS：判断有向图是否成环 → 三色标记法

53. 课程表 (No.207) MEDIUM

Solution 2 – 广度优先

思路

课程之间的关系其实是一个拓扑排序。用邻接表graph存储图，degree记录所有节点的入度，然后进行多源广度优先搜索，搜索的过程就是模拟一个课一个课修读的过程，队列queue中存的都是目前“可修读”的课程。

一开始入队的都是入度为0的没有先修的课程，遍历某个course的时候，其“后修”课程全部可修，加入队列，对应入度要减1。最终如果所有课入度减为0就成功修完，反之就有课修不了。

时间复杂度：$\mathcal{O}(|V|+|E|)$

空间复杂度：$\mathcal{O}(|V|+|E|)$

```python
class Solution(object):
    def canFinish(self, numCourses, prerequisites):
        """
        :type numCourses: int
        :type prerequisites: List[List[int]]
        :rtype: bool
        """
        # graph中key=课程编号,value=key的后修课程列表，degree存储入度
        graph, degree = defaultdict(list), [0] * numCourses
        for pre in prerequisites:
            graph[pre[1]].append(pre[0])
            degree[pre[0]] += 1
        # 把所有入度为0的课程加入queue（不需要先修），queue是可修课程队列
        queue = []
        for i in range(numCourses):
            if not degree[i]:
                queue.append(i)
        # 多源广度优先搜索
        while queue:
            course = queue.pop(0)  # 模拟把course修掉
            for post in graph[course]:  # 遍历course所有后修课程
                degree[post] -= 1
                if not degree[post]:  # 入度为0就可修了
                    queue.append(post)
        if any(degree):  # 有课程没被修，入度不为0
            return False
        return True
```

54. 实现 Trie 前缀树 (No.208) MEDIUM

Trie（发音类似 "try"）或者说 **前缀树** 是一种树形数据结构，用于高效地存储和检索字符串数据集中的键。这一数据结构有相当多的应用情景，例如自动补全和拼写检查。

请你实现 Trie 类：

- `Trie()` 初始化前缀树对象。
- `void insert(String word)` 向前缀树中插入字符串 `word` 。
- `boolean search(String word)` 如果字符串 `word` 在前缀树中，返回 `true`（即，在检索之前已经插入）；否则，返回 `false` 。
- `boolean startsWith(String prefix)` 如果之前已经插入的字符串 `word` 的前缀之一为 `prefix` ，返回 `true` ；否则，返回 `false` 。

示例：

> **输入**
> `["Trie", "insert", "search", "search", "startsWith", "insert", "search"]`
> `[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]`
> **输出**
> `[null, null, true, false, true, null, true]`
>
> **解释**
> ```
> Trie trie = new Trie();
> trie.insert("apple");
> trie.search("apple");   // 返回 True
> trie.search("app");     // 返回 False
> trie.startsWith("app"); // 返回 True
> trie.insert("app");
> trie.search("app");     // 返回 True
> ```

提示：

- `1 <= word.length, prefix.length <= 2000`
- `word` 和 `prefix` 仅由小写英文字母组成
- `insert`、`search` 和 `startsWith` 调用次数 **总计** 不超过 $3 * 10^4$ 次

54. 实现 Trie 前缀树 (No.208) MEDIUM

思路

字母前缀树其实是一个通过哈希表实现的26叉树。创建一个Node类包含next哈希表key=字母，value=如果下一个字符是key应跳转的节点；由于要区分前缀和单词本身，每个节点需要end表示其是否是词尾。知道是26叉树以后其他操作都不难了。

```python
class Trie:

    class Node:  # 内部类Node
        def __init__(self):
            self.next = {}  # key=字母, value=下一个节点
            self.end = 0  # 每个节点是否可以作为词尾符

    def __init__(self):
        self.root = self.Node()

    def insert(self, word: str) -> None:
        cur = self.root
        for char in word:
            if char not in cur.next:
                cur.next[char] = self.Node()
            cur = cur.next[char]
        # cur停在最后一个节点, end置1
        cur.end = 1
```

```python
class Trie:

    def search(self, word: str) -> bool:
        cur = self.root
        for char in word:
            # char成为下一个字符不合法
            if char not in cur.next:
                return False
            cur = cur.next[char]
        if not cur.end:  # 最后一个字符收尾不合法
            return False
        return True

    def startsWith(self, prefix: str) -> bool:
        cur = self.root
        for char in prefix:
            if char not in cur.next:
                return False
            cur = cur.next[char]
        return True


# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)
```